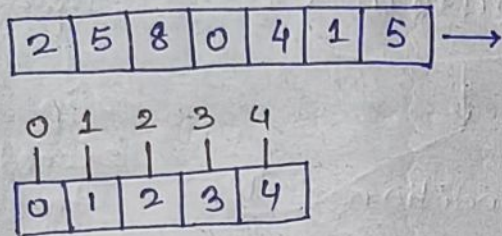
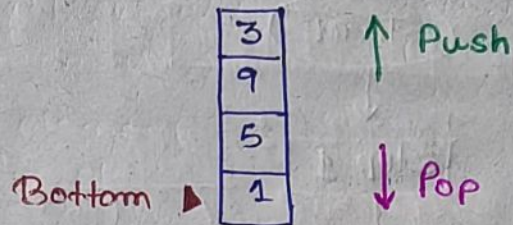


DATA STRUCTURE AND Algorithm - SHEET NOTES

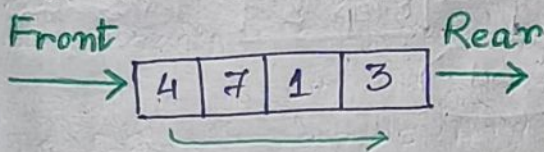
◆ Array



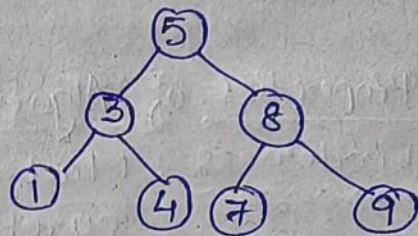
◆ Stack



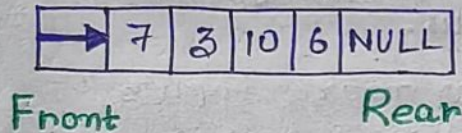
◆ Queue



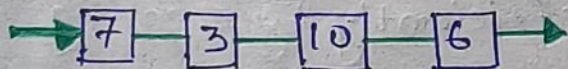
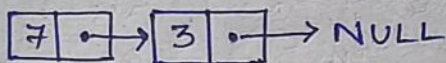
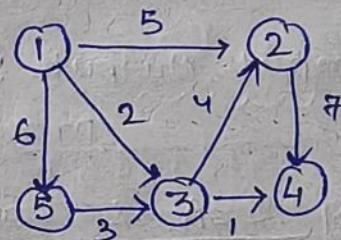
◆ Binary Tree



◆ Linked List



◆ Graph

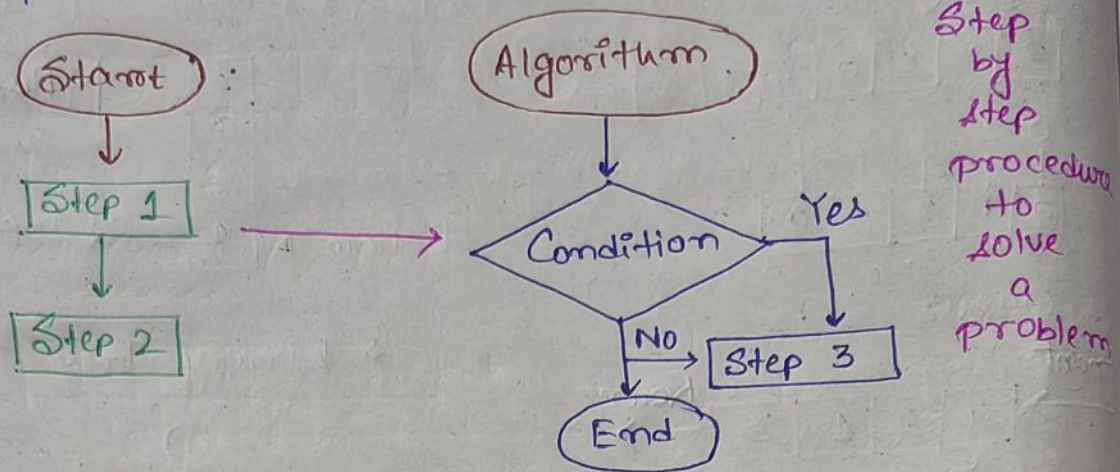


1. What is Data Structure?

A data structure is a way to store and organize data in a computer so that it can be used efficiently. Different data structures are designed for different kinds of data operations to make data processing more effective.

2. What is Algorithm?

An Algorithm is a finite sequence of well defined steps to solve a specific problem. It is like a recipe that defines the exact steps in a particular order.



* Properties of Algorithm

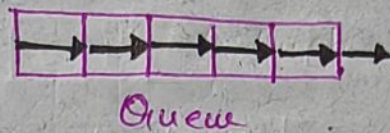
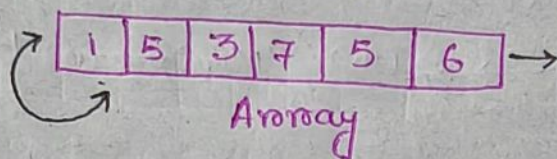
1. Finiteness : Must terminate after a finite number of steps.
2. Definiteness : Steps must be precisely defined and unambiguous.
3. Input Output : Take zero or more inputs, produce at least one output.

* Why DSA is important?

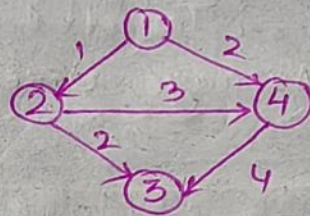
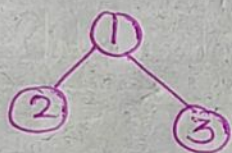
1. Efficient Problem Solving :- DSA provides tools to solve problems efficiently by choosing the right DSA.
2. Optimal Resource Usage :- Helps use memory and time effectively by ensuring minimal resource consumption.
3. Coding Interviews & competitions :- Crucial for coding interviews and programming contests as knowledge of DSA is essential for tackling challenging problems.

* Types of Data Structure

1. Linear :- Linear data structure organize data in a sequential manner where each element is adjacent to the previous one.



2. Non-linear :- Non-linear data structures organize data hierarchically, collecting elements in a more complex, branching manner.



* Abstract Data type (ADT)

ADTs are defined by their operations, not by their implementation.

Example include :

List

Stack

Queue

Map

Set

* Time Complexity & Space Complexity

• Why complexity Analysis?

1. Evaluate Efficiency :- Complexity analysis helps in measuring an algorithm's efficiency by evaluating its running time (time complexity) and memory usage (space complexity).
2. Compare Algorithms :- Helps in comparing different algorithms to choose the most optimal one for a given problem.
3. Scalability :- Ensures that our algorithm can handle large inputs efficiently without degrading.

Performance.

* Best, Average & Worst Case

Algorithms are evaluated based on their performance in different scenarios:-

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

 $\rightarrow \Omega(n)$ Minimum Tree

Target = 1

1. Best Case ($\Omega(n)$)

1	2	2	3	4	5	6	7
---	---	---	---	---	---	---	---

 $\rightarrow$

Minimum Tree

Target = 1

2. Average Case [$\Theta(n)$]

5	3	1	1	7	6	2	4	4
---	---	---	---	---	---	---	---	---

 $\rightarrow$

Expected time

Targets = 6

3. Worst Case [$O(n)$]

4	6	7	8	8	9	10	13
---	---	---	---	---	---	----	----

 $\rightarrow$

Maximum time

Target = 13

* Big-O, Big- Ω , Big Θ

◆ Why complexity analysis?

↑ Big-O	Upper Bound	$f(n) \leq O(g(n))$
↓ Big- Ω	Lower Bound	$f(n) \geq \Omega(g(n))$
↑ Big- Θ	Tight Bound	$f(n) = \Theta(g(n))$

* Space Complexity

◆ What is Space Complexity?

Space Complexity measures the amount of memory an algorithm uses during its execution.

- Input Size :- Memory needed to store the inputs.
- Aux. Space :- Extra memory needed apart
- Output Space :- Memory used to store the output data.

* Complexity of Common Loops & Recursion

◆ Complexity of Loops :-

1. Linear Loop :

```
for (int i=0; i<n; i++)
{ ... }
```

→ $O(n)$

2. Nested loops :

```
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++) {
        ....
    }
}
```

→ $O(n^2)$

3. Logarithmic loop :

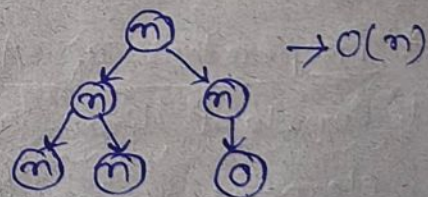
```
for (int i=1; i<n; i*=2)
{ .... }
```

→ $O(\log n)$

◆ Complexity of Recursion

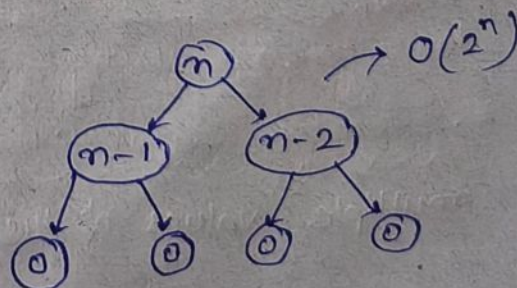
1. Linear Recursion

```
void recurse (int n) {
    if (n ≤ 0) { return; }
    recurse (n-1);
}
```



2. Binary Recursion

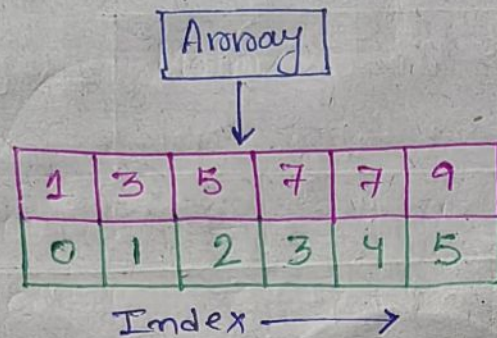
```
void recurse (int n) {
    if (n ≤ 0) { return; }
    recurse (n-1);
    recurse (n-1);
}
```



Chapter 2 : Arrays

* Introduction to Arrays

An array is a collection of elements of the same data type, stored in contiguous memory locations. It is a data structure that stores elements in a sequential manner.



* Characteristics of Array

1. Fixed Size :- The size of an array is defined at the time of declaration and does not change.
2. Homogeneous Elements :- All the elements in an array are the same data type.
3. Contiguous Memory Location :- Elements are stored in adjacent memory locations, allowing constant time access.

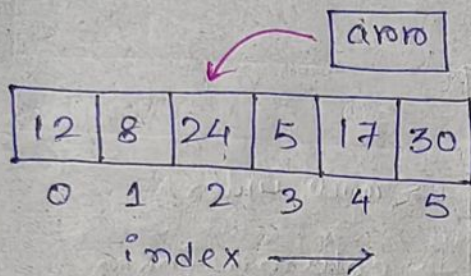
* Properties of Arrays

1. Constant time access :- Direct access to any element using its index.
2. Easy Traversal :- Simple to loop through all the elements using a for loop.
3. Effective Data Storage :- Useful for storing multiple values of the same type.

1D Arrays

* 1D Arrays

- A 1D array is a linear data structure that stores a collection of elements of the same data type in a single, contiguous block of memory.
- Each element in a 1D array is accessed using an index, which represents the element's position.



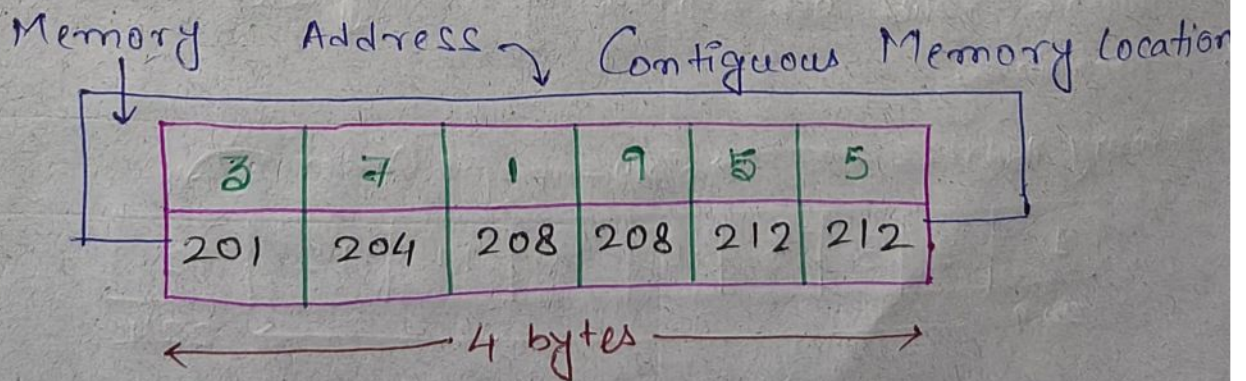
$arr[2] = 24$
(Accessed element)

* Declaration and initialization

- Syntax :- $int\ arr[5];$ Declares an integer of size 5 without initializing elements.
- Example :- $int\ arr[5] = \{3, 7, 1, 9, 5\};$ declares an integer array of size 5 and initializes it with the values 3, 7, 1, 9, 5.

* Memory Representation

- The integer array is stored in a contiguous block of memory. Each element takes 4 bytes of memory.



2D Arrays

* 2D Arrays

- A 2D array is a collection of elements arranged in rows and columns, forming a matrix like structure.
- It allows storing data in a tabular format, with elements accessible using a pair of indices.

Matrix

	0	1	2	3	
Row	0	1	2	3	4
Index	0	5	6	7	8
	0	9	10	11	12

→ row index

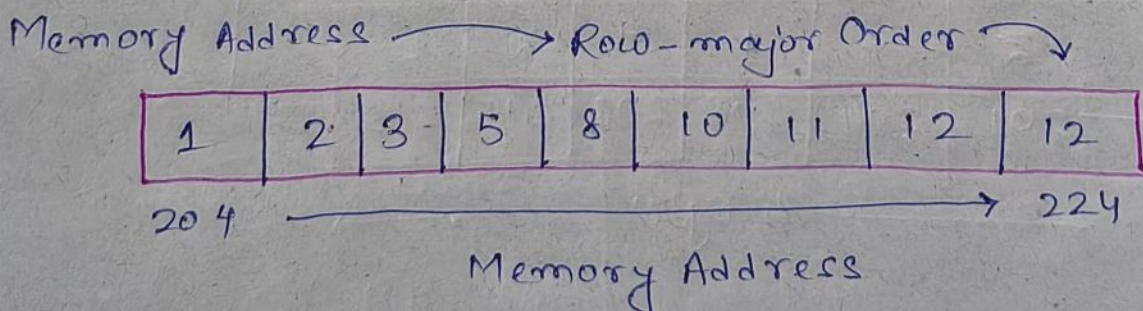
→ column index

* Declaration & Initialization

- Syntax :- `int matrix [3][4]` : Declares a 3x4 integers array without initializing elements.
- Examples :- `int matrix [3][4] = { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 };`

* Memory Representation

- The 2D array is stored in row-major order, where rows are stored in contiguous memory locations.

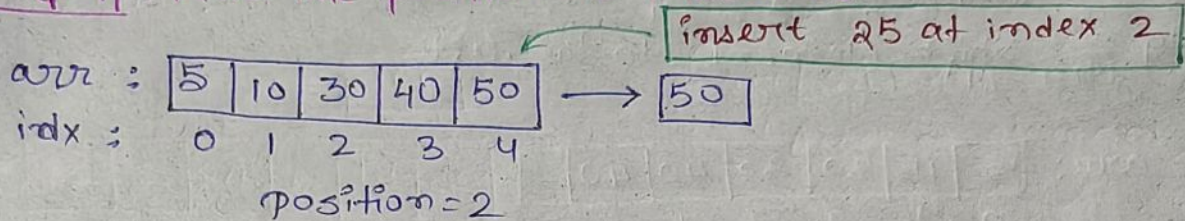


Array Operation

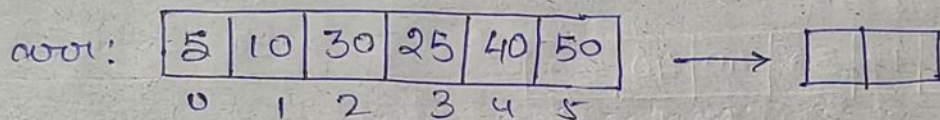
* Insertion

- Array insertion involves adding a new element to a specific position within an array.
- This shifts the subsequent elements one position to the right to make space for the new element.

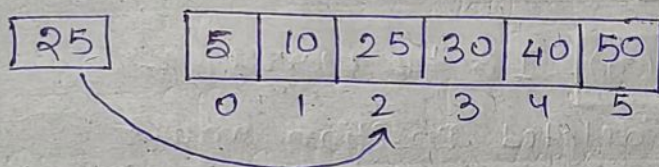
Step-1 : Find the position to insert at



Step-2 :- Shift elements to the right



Step 3 : Insert the new element at the found pos.



* Properties of insertion

1. Time Complexity :- $O(n)$: In the worst case, all elements may need to be shifted, making it linear time.
2. Insertion at the end :- $O(1)$: If there is space at no shifting needed.
3. Full Array :- Insertion is not possible when the array is full and has no space for more elements.

* Deletion

- Array deletion involves removing an existing element from a specific position within an array.
- To fill the gap, the subsequent elements are shifted one position to the left.

Step 1: Find the position to insert at

delete element at index 2

arr:

5	15	30	30	40
0	1	2	3	4

 $\rightarrow$

50

 pos = 2

Step 2: Shift element to the left

arr:

5	15	30	30	40	40	
0	1	2	3	4	5	

Step 3: fill the last position with a default value

25	5	15	30	40	40	50
0	1	2	3	4	5	6

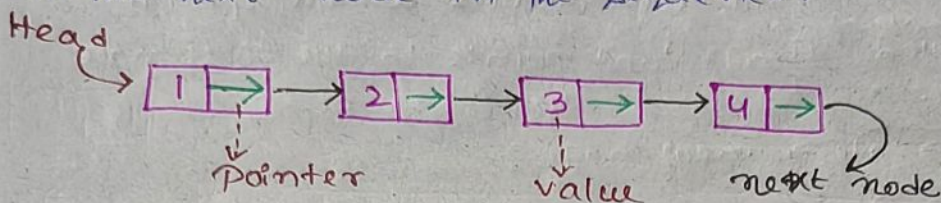
* Properties of Deletion

1. Time Complexity: - $O(n)$: In the worst case, all elements after the deleted position must be shifted.
2. Deletion from the end: - $O(1)$: If no default values is needed, as no shift is required.
3. Multiple Deletions: Multiple deletions may cause fragmentation, leaving empty slots in the array.

CHAPTER 3: LINKED LIST

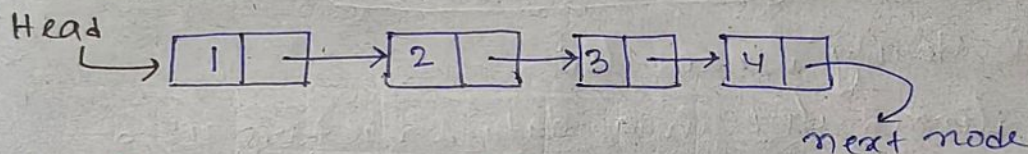
* Introduction

A Linked list is a data structure used for storing a sequence of elements. Unlike arrays, linked lists consist of nodes where each node contains a value to the next node in the sequence.



- Dynamic Size : Linked list can easily grow and shrink in size, making them suitable for applications where the number of elements is unknown or varying.

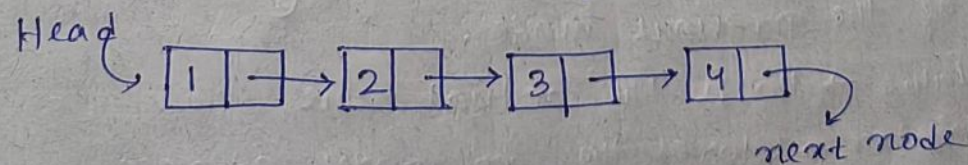
- Efficient Insertions/Deletions : Elements can be easily inserted or removed without the need to shift other elements, unlike arrays.



• Singly Linked List

♦ What is singly linked list?

A singly linked list is a data structure used for storing a sequence of elements where each element points to the next one through a pointer. It consists of a series of connected nodes, where each node contains a piece of data and a pointer to the next node in the sequence.



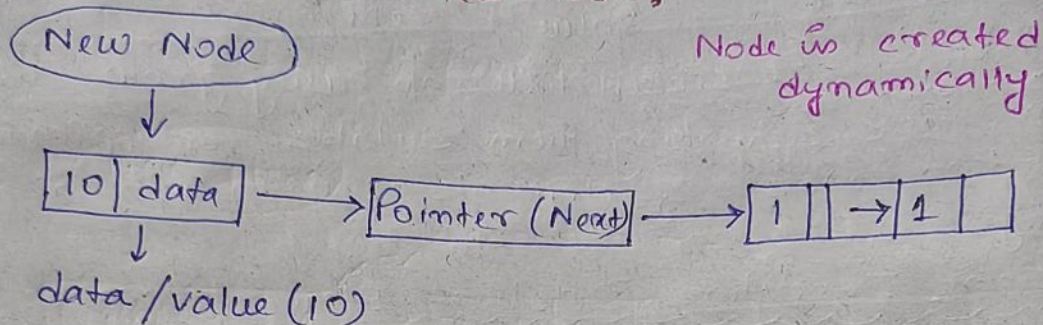
- Each node consists of two parts:

- ① Data/value : Holds the actual value. (eg. 10)
- ② Pointer : Stores the memory address of

the next node.

- Unidirectional :- It only allows traversal in one direction (from head to last node)
- Memory Efficient :- Nodes are dynamically created and memory is allocated as needed.
- Head Pointer :- The list starts with a special pointer called "Head" that points to the first node in the list.

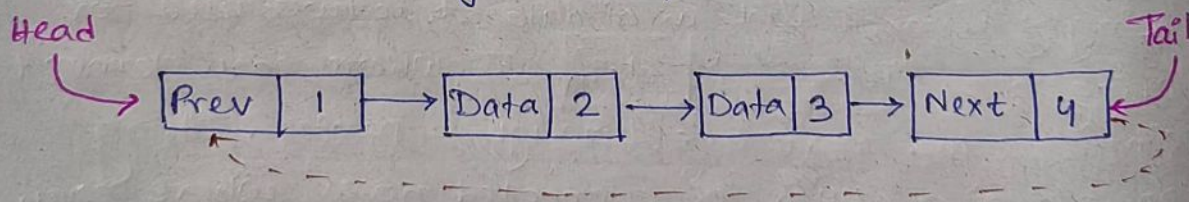
* How to create a new node?



Doubly Linked List

* What is doubly linked list?

A doubly linked list is a data structure used for storing a sequence of elements where each element is connected to both its previous and next nodes through two pointers.

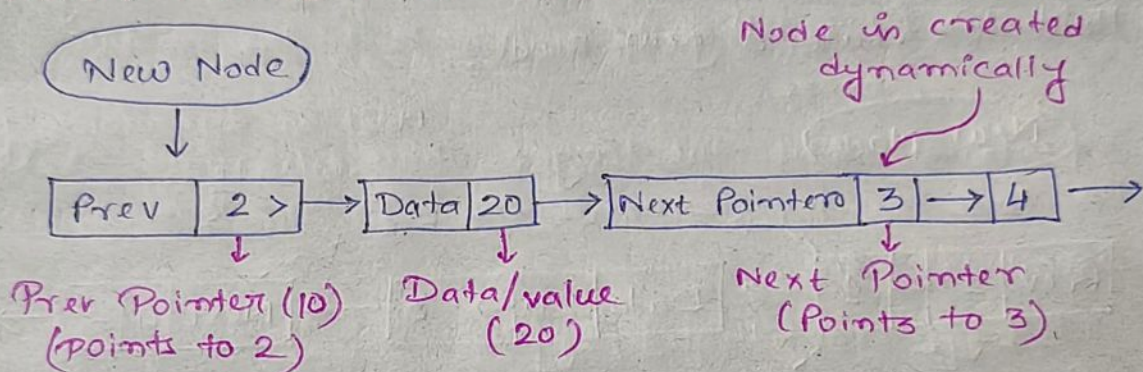


- Each node consists of three parts :-
 - ① prev pointer → Stores the memory address of the previous node.
 - ② Data/Value → Holds the actual value
 - ③ Next Pointer → Stores the memory address of the next node.

- Bidirectional :- Supports traversal in both forward and backward directions.
- More memory :- Requires extra memory for storing the previous pointer, unlike singly linked lists.
- Head & Tail Pointers :- The list has two special pointers :-

- ① Head points the first node.
- ② Tail points the last node.

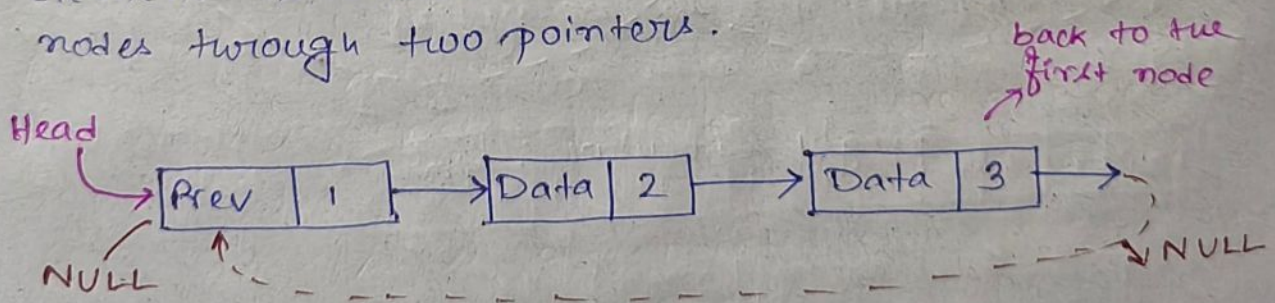
* How to create a new node?



Circular Linked List

◆ What is circular linked list?

A circular linked list is a data structure used for storing a sequence of elements where each element is connected to both its previous and next nodes through two pointers.



- Forms a circle : The last node points back to the first node.
- No Null value : Unlike linear linked list, there is no NULL pointers, in a circular linked list because it forms a continuous loop.

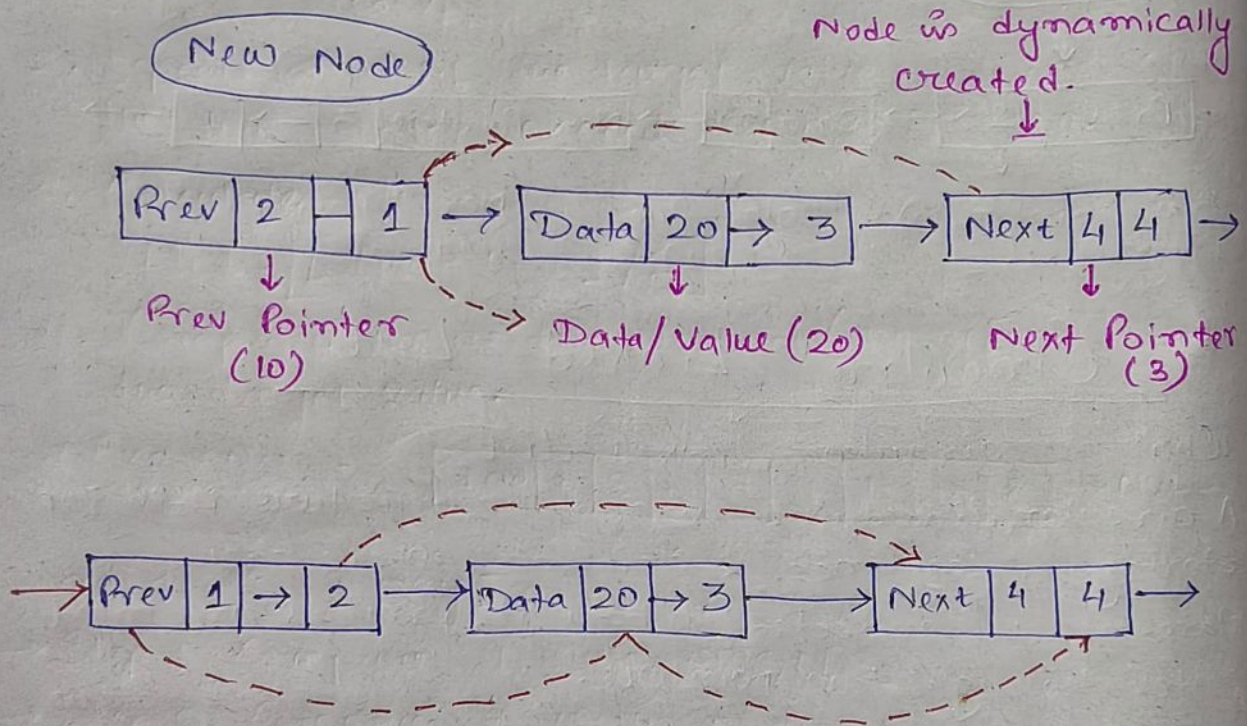
- Can be singly or doubly linked:

(i) Singly circular: Each node points to the next node and the last node points back to the first node.

(ii) Doubly circular: Each node points to both the next and previous nodes and the last node points back to the first node.

- Head Pointer: Points to the first node in the list.

* How to create a new node?



Singly Circular : List

CHAPTER - 4 : SEARCHING

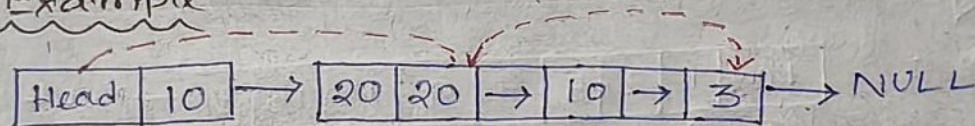
1. Linear Search

Linear search is a simple searching algorithm that checks each element of a list sequentially until the desired element ~~of a list~~ is found.

* Steps

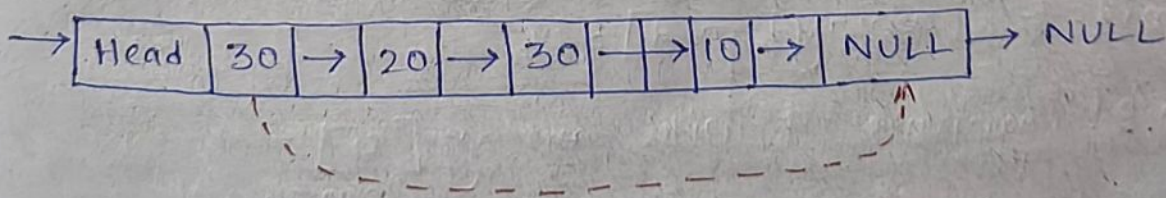
- ① Start from the first element
 - Prev : initially set to NULL
 - Current : Initially set to head
 - Next : Points to the node after current
- ② Reverse pointers one by one
- ③ Move prev, current and Next pointers one step forward
- ④ Repeat until current becomes NULL.

* Example



- Start with the first element and compare each element with the target (7).
 - When element 7 is found at index 2, return 2
- Result: 2

* Target = The value were searching for "7".



2. Binary Search

Binary Search is an efficient searching algorithm that checks each element of a list sequentially until the desired element is found. It works with both unsorted and sorted arrays.

* Steps

1. Initialize three pointers:
 - Low $\rightarrow$ Points to the first index of the array.
 - High $\rightarrow$ Points to the last index of the array.
 - Mid $\rightarrow (Low + High) / 2$, points to the middle index.
2. Repeat ~~un~~ until the target value is found, or the low pointer exceeds the high pointers.
3. Calculate Mid:
$$Mid = \lfloor [Low + High] / 2 \rfloor$$
4. Compare the middle element ($array[mid]$) with the target value.

* Examples :-

Low	1	3	5	7	9	11	15	8	$\rightarrow$ NULL
	0	1	2	3	4	5	6	7	

mid = 3 $array[mid] = 7$

$\rightarrow$ Step 1: Compare 9 with $array[3]$

$\rightarrow$ Step 2: Compare 9 with $array[5]$

Low	1	3	5	7	9	11	15	5	19	8	6	
	0	1	2	3	4	5	6	6	6	7	7	

mid = 5, $array[mid] = 11$

$\swarrow$ search right half

$\rightarrow$ Step 3: Compare 9 with $array[4]$

Low	1	3	5	7	9	11	15	15	8	6	
	0	1	2	3	4	5	6	6	7	7	

$\swarrow$ search left half

mid = 4, $array[mid] = 9$

CHAPTER- 5 : SORTING

Introduction to sorting algorithms

Sorting Algorithms are methods used to rearrange elements in a list or an array into a specific order. They are fundamental in computer science for organizing data efficiently.

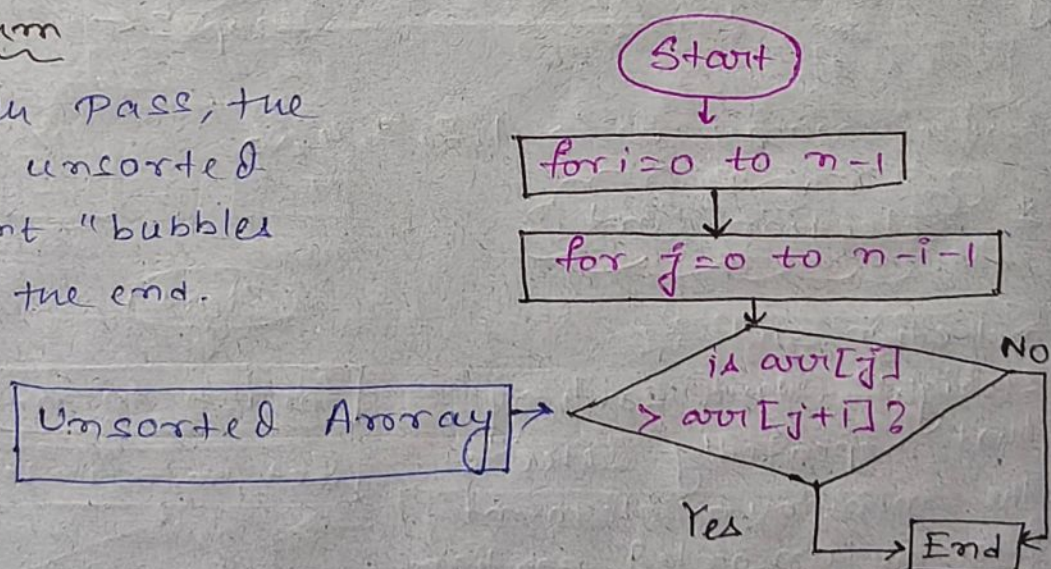
- | | |
|-------------------|------------------|
| 1. Bubble Sort | 6. Heap Sort |
| 2. Insertion Sort | 7. Counting Sort |
| 3. Selection Sort | 8. Radix Sort |
| 4. Merge Sort | 9. Stability & |
| 5. Quick Sort | in-Place sort |

* Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the entire list is sorted.

Algorithm

For each Pass, the largest unsorted element "bubbles up" to the end.



Examples

unsorted array = [4, 2, 6, 3, 1]

Pass 1: (i=0)

$j=0 \rightarrow 2, 4, 6, 3, 1 \rightarrow$ Since $4 > 2$, swap $4 \leftarrow 2$

$j=1 \rightarrow 2, 4, 6, 3, 1 \rightarrow$ No swap

$j=2 \rightarrow 2, 4, 3, 6, 1$

$\rightarrow$ since $6 > 3$
swap $6 \leftrightarrow 3$

$j=3 \rightarrow 2, 4, 3, 1, 6$

$\rightarrow$ since $6 > 1$
swap $6 \leftrightarrow 1$

Pass 2 : ($i=1$)

Already found the largest element to end

$2, 4, 3, 1, 6$

Pass 3 : ($i=2$)

$j=0 \rightarrow 2, 4, 3, 1, 6$

$\rightarrow$ no swap

$j=1 \rightarrow 2, 3, 4, 1, 6$

$\rightarrow$ since $4 > 3$,
swap $4 \leftrightarrow 3$

$j=2 \rightarrow 2, 3, 1, 4, 6$

$\rightarrow$ since $4 > 1$
swap $4 \leftrightarrow 1$

Pass 4 : ($i=3$)

$j=0 \rightarrow 2, 3, 1, 4, 6$

$\rightarrow$ no swap

$j=1 \rightarrow 2, 1, 3, 4, 6$

$\rightarrow$ since $3 > 1$,
swap $3 \leftrightarrow 1$

Pass 5 : ($i=4$)

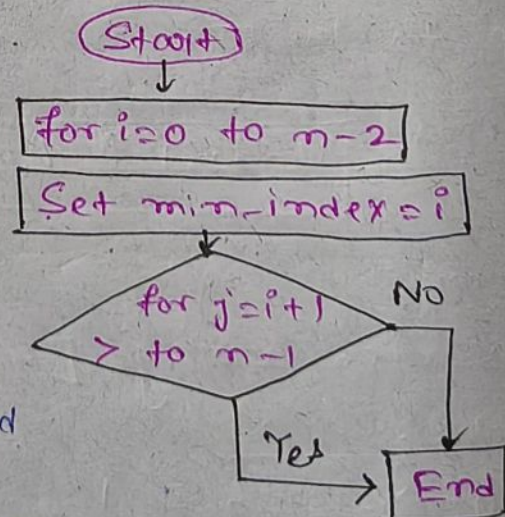
$j=0 \rightarrow 1, 2, 3, 4, 6$

$\rightarrow$ since $2 > 1$,
swap $2 \leftrightarrow 1$

Sorted array is $\rightarrow 1, 2, 3, 4, 6$

* Selection Sort

Selection sort is a straightforward sorting that repeatedly selects the minimum element from the unsorted portion of the list and swaps it with the first unsorted element. This process is repeated until the entire list is sorted.



Example

unsorted array = $[8, 5, 2, 9, 5]$

minimum = 2

Pass 1: ($i=0$)

2, 5, 8, 9, 5 $\rightarrow$ swap 8 & 2

Pass 2: ($i=1$)

2, 5, 8, 9, 5 $\rightarrow$ no swap

Pass 3: ($i=2$)

2, 5, 5, 9, 8 $\rightarrow$ swap

Pass 4: ($i=3$)

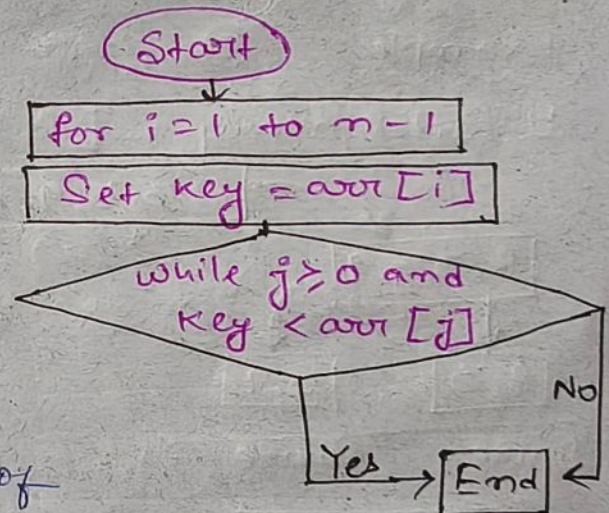
2, 5, 5, 8, 9 $\rightarrow$ swap

Pass 5: ($i=4$)

2, 5, 5, 8, 9 $\rightarrow$ Sorted array

* Insertion Sort

Insertion sort is a simple sorting algorithm that builds the final sorted list one element at a time. It takes each element from the unsorted part and inserts it into the correct position in the sorted part of the list. This process is repeated until the entire list is sorted.



Example

Unsorted array = $[7, 4, 5, 8, 9]$

Pass 1: ($i=0$)

4, 7, 5, 8, 9 $\rightarrow$ since $7 > 4$, swap $7 \leftarrow 4$

Pass 2: ($i=1$)

4, 5, 7, 8, 9 $\rightarrow$ since $7 > 5$, swap $7 \leftarrow 5$

Pass 3 (i=2)

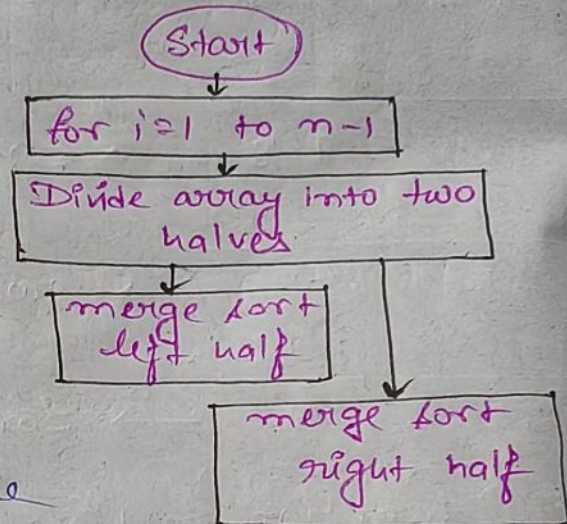
4, 5, 7, 8, 9 $\rightarrow$ no swap

Pass 4 (i=3)

4, 5, 7, 8, 9 $\rightarrow$ no swap

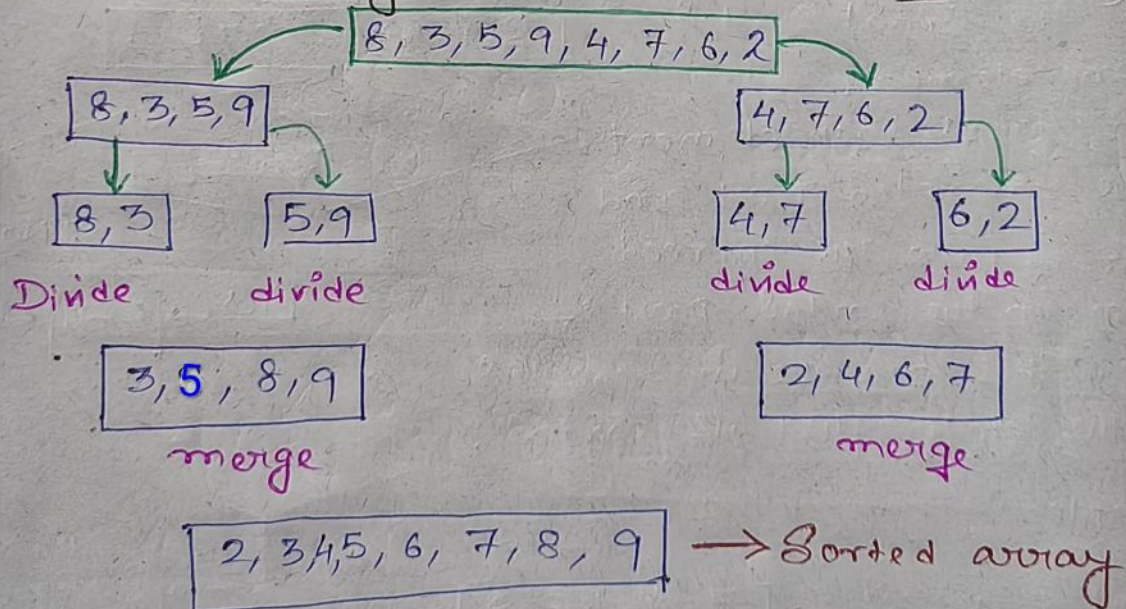
* Merge Sort

Merge sort is an efficient sorting algorithm based on the divide and conquer strategy. It works by recursively dividing the list into two halves until each half contains a single element. Then it merges the halves back together in a sorted manner.



Example

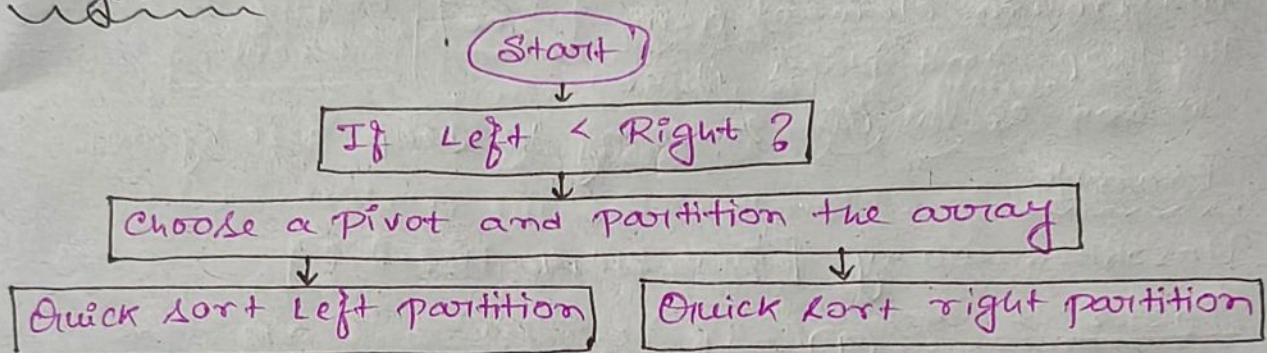
Unsorted array = [8, 3, 5, 9, 4, 7, 6, 2]



* Quick Sort

Quick Sort is an efficient sorting algorithm based on the Divide and Conquer strategy. It works by selecting a pivot element and partitioning the list into elements ^{less} than or equal to the pivot and elements greater than the pivot. The process is recursively applied to the sublists.

Algorithm



Example

Unsorted array = [7, 2, 1, 6, 8, 5, 3, 4]

$i = 4$

Pass 1: ($i = 4$)

Smaller than 4 $\rightarrow$ 2, 1, 3

Larger than 4 $\rightarrow$ 7, 6, 8, 5

array = { 2, 1, 3, 4, 8, 5, 7, 6 }

Pass 2: ($i = 3$) [left sub array]

Smaller than 3 $\rightarrow$ 2, 1

Sub array 1, 2, 3 [$\because 2 > i$] swap

Pass 3: [Right sub array]

Compare {8, 5, 7} to 6

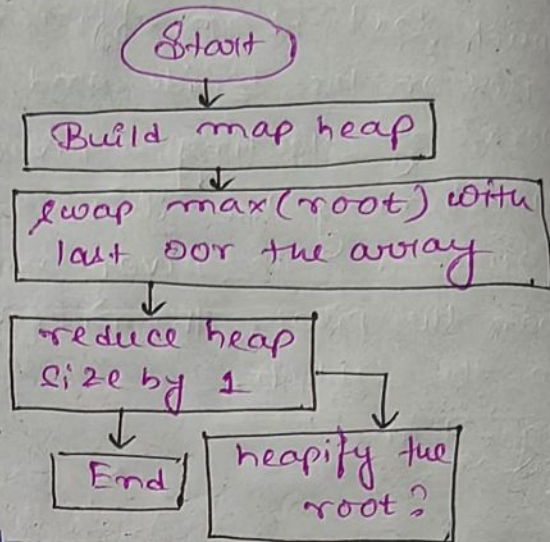
Sub array 5, 6, 7, 8

Pass 4: (Sorted array)

1, 2, 3, 4, 5, 6, 7, 8

*Heap Sort

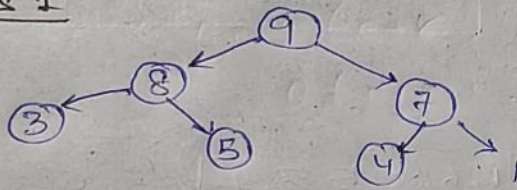
Heap sort is a comparison based sorting algorithm that uses a binary heap data structure to arrange elements in order. It first builds the max heap from the list, then repeatedly removes the maximum element from the heap and places it at the end of the list, reducing the heap size by one.



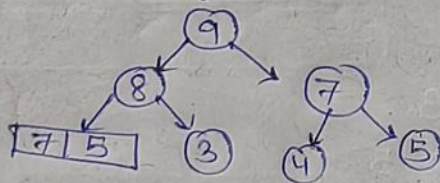
Example

Unsorted array = [3, 9, 2, 1, 4, 5, 7, 8, 8]

Pass 1



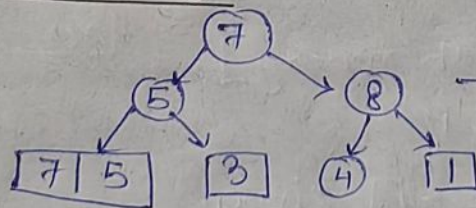
Pass 2: Heapify



(swap 9 with)

sorted list [8, 9]

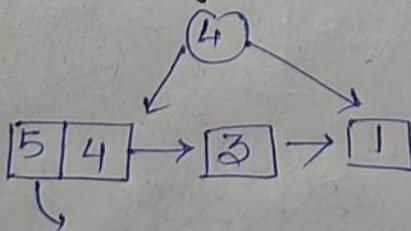
Pass-3: Heapify



7, 7, 8, 4, 8, 9

sorted list [8, 9]

Pass 4: Heapify



sorted list

[5, 7, 8, 9]

max heap

Sorted array → 1, 2, 3, 4, 5, 6, 7, 8, 9

* Counting Sort

Counting Sort is a non-comparison based sorting algorithm that works by counting the occurrence of each unique element in an input list. It creates a count array to store the frequency of each element. Using this count array, it then calculates the positions of each element in the sorted output array. Counting Sort is efficient for sorting integers within a known limited range.

Example

Unsorted array: $[2, 1, 2, 3, 1, 2, 4, 3, 3, 2]$

	2	4	3	1
0	1	2	3	4

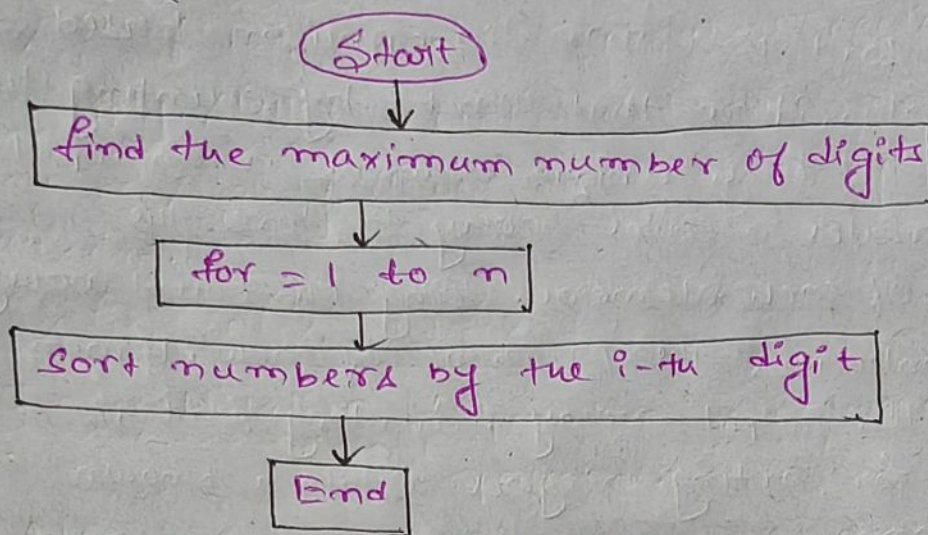
1 1 2 2 2 3 3 3 4 $\rightarrow$ Sorted array

[Depends on the occurrence of element]

* Radix Sort

Radix Sort is a non-comparison based sorting algorithm that sorts numbers by processing individual digits. It works by sorting the numbers digit by digit, starting from the least significant digit to the most significant digit (LSD to MSD). At each digit, counting sort is utilized as a stable sorting subroutine to sort the numbers. Radix sort is effective for sorting integers and can also be adapted for fixed length strings.

Algorithm



* Stability & in place sorting

- Stability : Maintenance the relative order of equal elements.
- Complexity : $O(n+k)$: Best and avg case
 $O(n+k)$: Worst case

CHAPTER - 6 : STACK

* What is Stack?

A Stack is a linear data structure that follows the Last In, First Out (LIFO) principle. This means that the last element added to the stack will be the first to be removed.

* Stack concept

LIFO

⇒ The last item pushed onto the stack is the first item to be popped off.

Push :- Adding an element to the top of the stack.

Pop :- Removing an element from the top of stack.

Example

History: 10, 20, 30, 30, 40, 10

10 → 20 → 30 → 30 → 40 → 10 →

* How to implement a stack?

Stacks can be implemented in two main ways: using an array and using a linked list.

• Using array

(i) fixed size :- uses a fixed size array to store the stack elements.

(ii) Simple and fast :- Easy to implement, requires less memory and provides fast access times.

(iii) Risk of overflow :- Stack overflow can occur if stack becomes full and no more elements can be pushed.

• Using Linked List

(i) Dynamic size :- Dynamically allocates memory as needed, grows and shrinks as elements are added or removed.

(ii) Memory Efficient: No overflow issue as the size of the stack can grow as needed.

(iii) Slightly Slower: - Linked list implementation may be slightly slower due to memory allocation overhead.

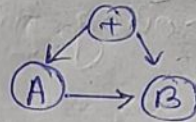
* Infix, Prefix, Postfix

(i) Infix Notation: - Operation is between the operands.

$A+B \longrightarrow A+B \longrightarrow A+B$

(ii) Prefix Notation: - Operator is before the operands.

$+AB$



(iii) Postfix Notation: - Operator is after the operands.

$AB+ \longrightarrow AB+ \longrightarrow AB+$

* How to evaluate an expression using stack?

Example $\rightarrow 3+2 \times 2 \rightarrow 3 \ 2 \ 2 \ \times \ +$

Step	Symbol	Stack
1	push 3	3
2	push 2	2 3
3	push 2	2 2 3
4	Apply $\times : 2 \times 2 = 4$	4 3
5	Apply $+$: $3 + 4 = 7$	7

final result : 7

CHAPTER-7 : QUEUE

* What is a Queue?

A Queue is a linear data structure that follows the FIFO (first in first out) principle.

This means the element inserted first will be removed first.

Queue Concepts

- Insertion happens from the rear.
- Deletion happens from the front.

* Basic Operations

- (i) Enqueue :- Insert an element at the rear.
- (ii) Dequeue :- Remove an element from the front.
- (iii) Front / Peek :- View the front element.
- (iv) IsEmpty :- Check if queue is empty
- (v) IsFull :- Check if queue is full

* Simple Queue

A Simple Queue is the basic implementation of a queue where :

- Elements are inserted from the rear.
- Elements are removed from the front.

Characteristics

- Follows FIFO
- Fixed size
- Wastage of memory can occur.

Limitation

Once elements are removed from the front, the empty space cannot be reused, which leads to false overflow.

Circular Queue

A circular queue is an improved version of a simple queue where the last position is connected back to the first position.

* Why circular queue?

- (i) Overcomes the memory wastage problem of simple queue.
- (ii) Reuses empty spaces efficiently.

■ Characteristics:

- (i) Front and rear move circularly.
- (ii) Last index is connected to first index.
- (iii) Better memory utilization.

Condition to move circularly:-

$$(\text{index} + 1) \% \text{Size}$$

Priority Queue

A priority queue is a special type of queue where each element has a priority. Elements with higher priority are dequeued before elements with lower priority. If two elements have same priority, they are dequeued in FIFO order.

■ Characteristics:

- (i) Each element has a priority.
- (ii) Element with higher priority is dequeued first.
- (iii) If priority is same $\rightarrow$ FIFO order is followed.

■ Use:-

- CPU scheduling

Deque (Double Ended Queue)

A deque is a queue in which insertion and deletion can be done from both ends.

■ Types

i) Input restricted deque

- insertion allowed at one end only
- deletion allowed at both ends.

ii) Output restricted deque

- insertion allowed at both ends.
- deletion allowed at one end only.

Applications

- undo/redo operations
- sliding window problems.

Queue Using Stack

i) Two stacks are used to implement a queue.

ii) Stack follows LIFO, but logic is applied to achieve FIFO.

Stack using Queue

i) Two queues are used to implement a stack.

ii) Queue follows FIFO, but logic is applied to achieve LIFO.

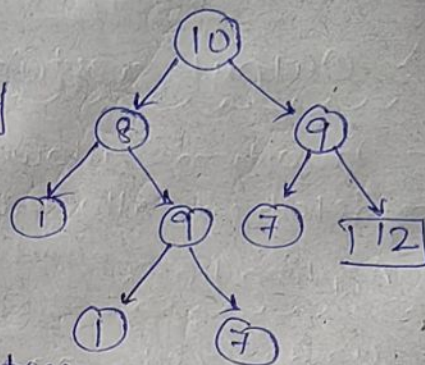
CHAPTER-8: TREES

* What is a Tree?

A tree is a non-linear data structure that represents data in a hierarchical structure. It consists of nodes connected by edges, with a special node called the root.

Key Points

- i) Organizes data hierarchically
- ii) no cycles
- iii) One root node.



Tree terminology

- 1) Node → Each element in a tree
- 2) Root → Topmost node of the tree
- 3) Parent → Node that has children
- 4) Child → Node derived from a parent
- 5) Leaf node → node with no children
- 6) Edge → Connection between two nodes
 - i) level → distance from root
 - ii) height → longest path from root to a leaf
- 7) Depth → Distance of a node from root
- 8) Subtree → Tree formed by a node and its descendants.

Binary Tree

A binary tree is a tree in which each node has at most two children:

- (i) left child
- (ii) Right child

* Properties

- i) maximum 2 children per node
- ii) Children are ordered.
- iii) can be empty

* What is a binary search tree?

A binary search tree is a binary tree that follows a specific order.

- 1. Left subtree $\rightarrow$ values less than root
- 2. Right subtree $\rightarrow$ values greater than root

* Advantages:-

- i) faster searching
- ii) Efficient insertion & deletion.

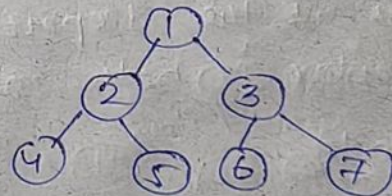
Tree Traversals

Tree Traversal means visiting all nodes of a tree exactly once in a specific order.

1. Inorder Traversal ($L \rightarrow \text{Root} \rightarrow R$)

Step:-

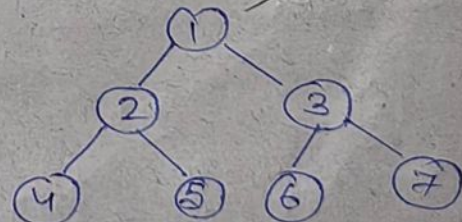
- 1. visit left subtree.
- 2. visit root
- 3. visit right subtree.



4, 2, 5, 1, 6, 3, 7

2. Preorder Traversal ($\text{Root} \rightarrow L \rightarrow R$)

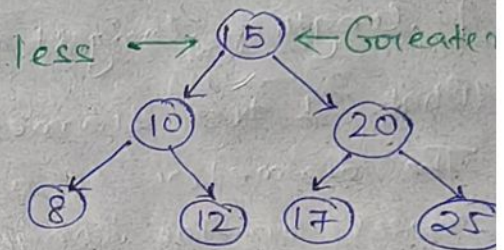
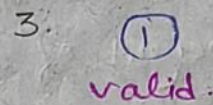
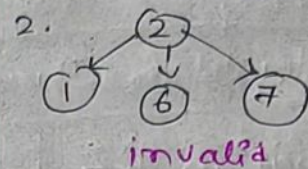
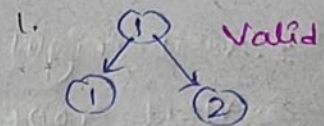
- 1. visit Root
- 2. visit left subtree
- 3. visit right subtree



1 2 4 5 3 6 7

3. Postorder Traversal ($L - R \rightarrow \text{Root}$)

- 1. visit left subtree



2. visit right subtree
3. visit root

4, 5, 2, 6, 7, 3, 1

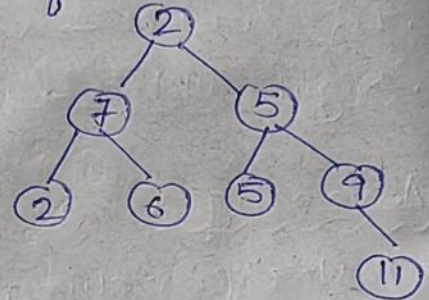
Height & Diameter

*What is tree height?

The height of a tree is the length of the longest path from the root node to a leaf node.

$$\therefore \text{Height} = 3$$

[length of longest path]



*What is tree diameter?

The diameter of a tree is the number of nodes on the longest path between any two leaf nodes.

$$\text{diameter} = 5$$

Lowest Common Ancestor

The lowest common ancestor (LCA) of two nodes is the deepest node that is a common ancestor of both nodes.

Example

- The LCA of nodes 5 and 11, is 7 is the LCA
- 5 is an ancestor of 5, and 11. So, 7 is the LCA.

CHAPTER-9: HEAPS

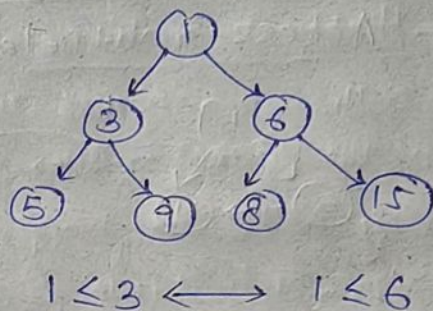
* What is a Heap?

A heap is a completely binary tree that follows a specific heap property.

* Types

i) Min heap

- The minimum element is always at the root
- Parent node value $\leq$ children values
- Root = smallest element.

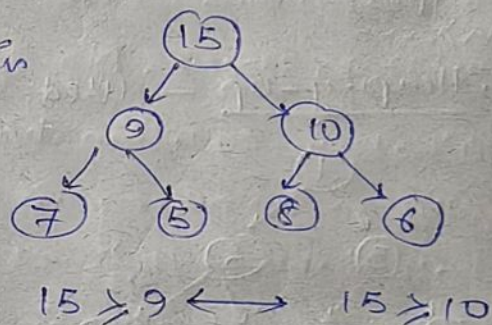


Use

- i) priority queue (min priority)
- ii) Dijkstra's algorithm

ii) Max heap

- The maximum element is always at the root
- Parent node values $\geq$ children values
- Root = largest element

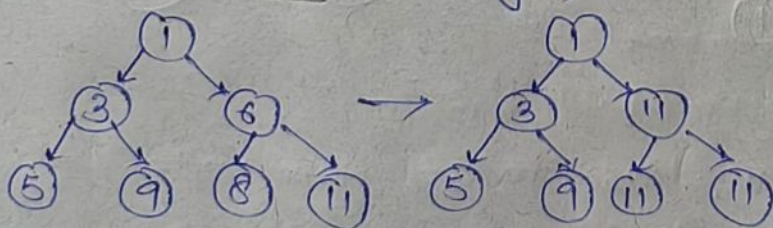


Use

- i) Heap sort
- ii) priority queue (max priority)

Heap operation

1. Insertion : $O(\log n)$

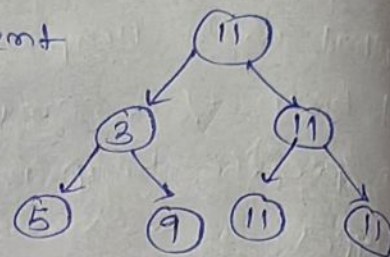


(i) Insert element at the end

(ii) Restore heap property using heapify-up

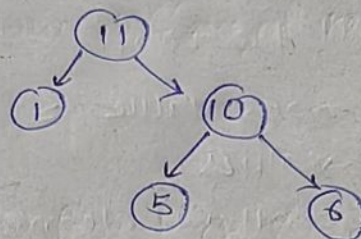
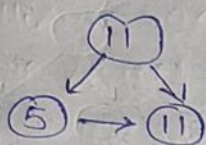
2. Deletion (Remove Root) : $O(\log n)$

1. Replace root with last element
2. Reduce heap size
3. Restore heap property using heapify-down.



3. Peek (Get min/max) : $O(1)$

- Access root element

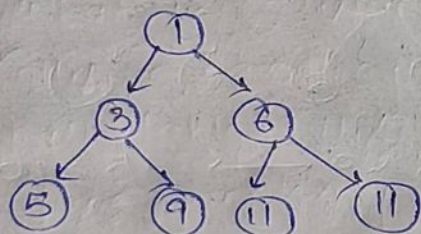


Heapify

- Keep heap valid
- efficient rearranging in $O(\log n)$ time

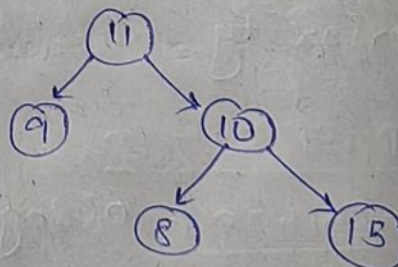
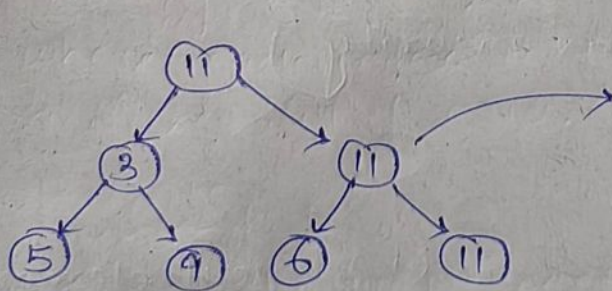
Types

1. Heapify-up - used after insertion



Time complexity
= $O(\log n)$

2. Heapify-down - Used after deletion



Priority Queue using Heap

* What is a priority Queue?

A priority Queue is an abstract data type where each element has a priority.

- Elements with highest / lowest priority are served first.

* Priority Queue using heap

Heap is an efficient way to implement a priority queue.

Min Heap

Heap is an efficient way to implement a priority queue.

- Lowest priority first
- Use min heap.

<u>Min heap</u>	
<u>Priority</u>	<u>Element</u>
1	(10) (10)
2	(20) (20)
3	(30)



<u>Max heap</u>	
<u>Priority</u>	<u>Element</u>
3	(30) (10)
2	(20) (10)
1	(10) (15)

* Application

i) Scheduling tasks

ii) Dijkstra's algorithm

iii) Load balancing.

CHAPTER - 10 : HASH TABLE

* What is Hash Table?

→ A hash table is a data structure that stores data in key-value pairs using a hash function.

- It allows fast insertion, deletion and search.
- Time complexity is $O(1)$

* Hash Function

A collision occurs when two different keys map to the same index.

◆ Why collision happen?

- | | |
|------------------------|---------------------------|
| (i) Limited table size | (iii) Poor hash function |
| (ii) Fast to compute | (iv) Minimizes collisions |

* Properties:

- (i) fast: Average $O(1)$ time complexity
- (ii) Dynamic: Size of table grows as needed
- (iii) Unordered: No specific order of element.

* What is collision handling?

Collision handling is the process of resolving hash table collisions

- Time complexity is $O(1)$

◆ Chaining

Chaining is a collision resolution technique where each index in the hash table points to a linked list of nodes.

■ Benefits

- 1) Simple to implement
- 2) Efficiency use in table size
- 3) No overflow: Can store more elements than table size.

* Open Addressing

All elements are stored inside the hash table itself.

- Time complexity $\rightarrow O(1)$

Types

① Linear probing

- $\text{index} = (\text{hash}(\text{key}) + i) \% \text{table size}$
- checks next slot sequentially

use $\rightarrow$ (i) easy to implement

(ii) Leads to clustering

② Quadratic probing

- $\text{index} = (\text{hash}(\text{key}) + i^2) \% \text{table size}$
- uses quadratic formula to reduce clustering

③ Double Hashing

- $\text{index} = (\text{hash1}(\text{key}) + i + \text{hash2}(\text{key})) \% \text{table size}$
- best collision distribution,

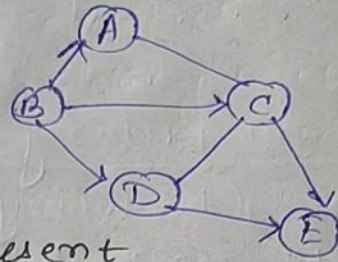
0	1	0	0	1	0
0	1	1	0	0	0

CHAPTER-11: GRAPHS

What is a Graph?

A Graph is a non linear data structure that consists of:

- (i) Vertices (Nodes)
- (ii) Edges (Connections)



Graphs are used to represent real world relationships like:

- (i) Social networks
- (ii) Maps & routes
- (iii) Computer Networks

Graph representation

◆ Adjacency matrix

(i) Row and column represent vertices

(ii) value 1 means edge exists

(iii) value 0 means no edge

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	0	1	1	1	0

properties:

- (i) uses more memory
- (ii) fast edge lookup
- (iii) Best for dense graph

◆ Adjacency list

Each vertex with a list of its connected vertices

Properties:

- (i) Memory efficient
- (ii) Slower edge lookup
- (iii) Best for sparse graph.

Breadth first Search (BFS)

BFS is a graph traversal algorithm that :

(i) visits nodes level by level

(ii) Uses Queue

(iii) Best for finding shortest path in unweighted graph.

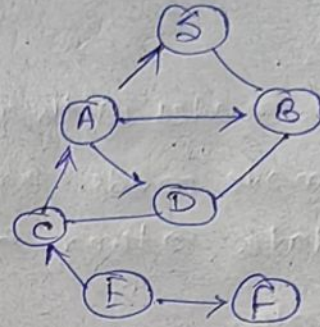
Steps

1. Start from source node

2. Visit all adjacent node

3. Move to the next level

→ BFS order :- $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$



Depth first Search (DFS)

(i) Goes deep before wide

(ii) Uses Stack / Recursion

(iii) Good for cycle detection

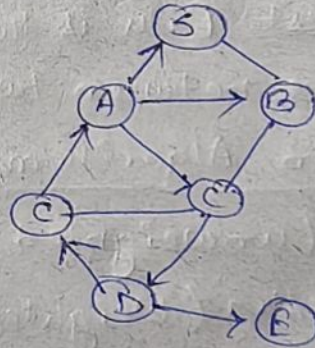
Steps

1. Start from a node

2. Visit an adjacent node

3. Backtrack if no nodes left

→ DFS order :- $S \rightarrow A \rightarrow C \rightarrow D \rightarrow E \rightarrow B$



Connected components

Every vertex is reachable from any other vertex in the same group.

Use

(i) Count isolated group

(ii) Check group connectivity

Example

